Compiling and Executing C++ Programs

Compiling is the way toward making an interpretation of source code into the local language of the PC's processor. The PC's processor works at an extremely basic level, executing programs in what is called machine language. This is a numeric code that portrays extremely little activities, for example, "include this byte," "point to this area in memory," or "duplicate this byte". Each of these instructions is expressed in binary which were hard to write. This issue was overwhelmed by the appearance of assembly language, which supplanted the numeric codes with (marginally) simpler to utilize character mnemonics, for example, CPY (for duplicate) and MOV (for move). Programs written in assembly language are processed into machine language by a program called an assembler.

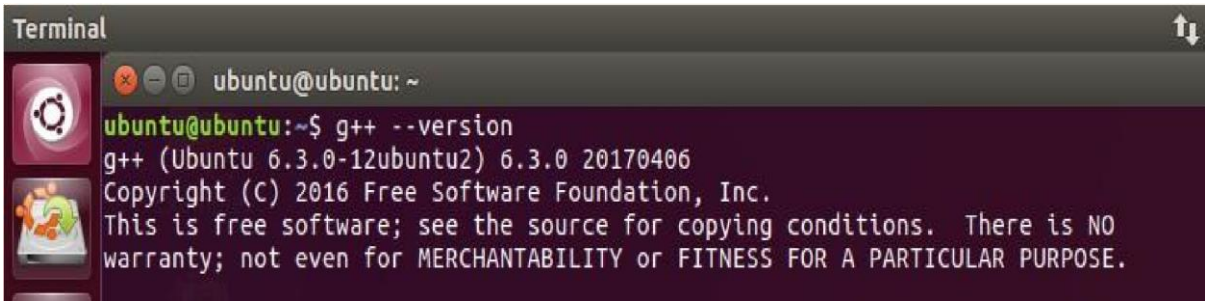
We next come to what are called high-level languages. They are called this since they enable the developer to be less worried about the details of what the processor is doing and more with taking care of the current issue. Programs written in high-level programming languages are converted into machine language by processing them with another program, called a compiler.

C++ compiler for Linux

C++ is a general-purpose programming language and widely used nowadays for competitive programming. It has imperative, object-oriented and generic programming features. C++ runs on lots of platform like Windows, Linux, Unix, Mac etc. Before we start programming with C++. We will need an environment to be set-up on our local computer to compile and run our C++ programs successfully. GNU C++ Compiler (g++) is a compiler in Linux which is used to compile C++ programs. It compiles both files with extension .c and .cpp as C++ files.

Installing g++ compiler

By default, g++ is provided with most of the Linux distributions. We can find the details of installed g++ compiler by writing the following command:



If g++ is not installed on your system; then it can be installed by writing the following commands

```
$ sudo apt-get update
```

```
$ sudo apt install g++
```

Compiling C++ program

We can compile a C++ program using the following command

```
$ sudo g++ source-file.cpp
```

The above command will generate an executable file a.out. We can use -o option to specify the output file name for the executable.

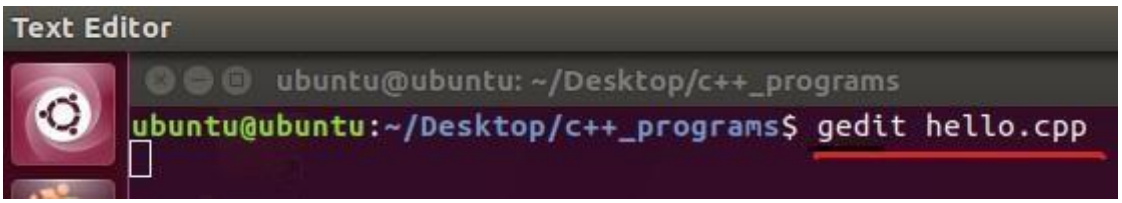
```
$ sudo g++ source-file.cpp -o executable-file
```

Running a C++ program

Onc a C++ program is successfully compiled, we can execute the created executable file by using the following command:

```
$ ./executable-file
```

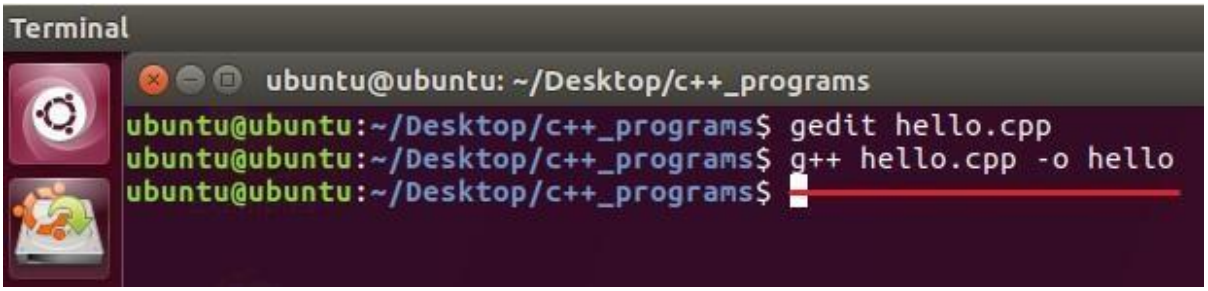
In the following example, we write a simple C++ program that displays a Hello World message and the then compile and execute this program using the above commands. To write the program, we use the gedit text editor. The source code file is saved with .cpp extension. Open the gedit editor and pass the name of the file (hello.cpp) to be created



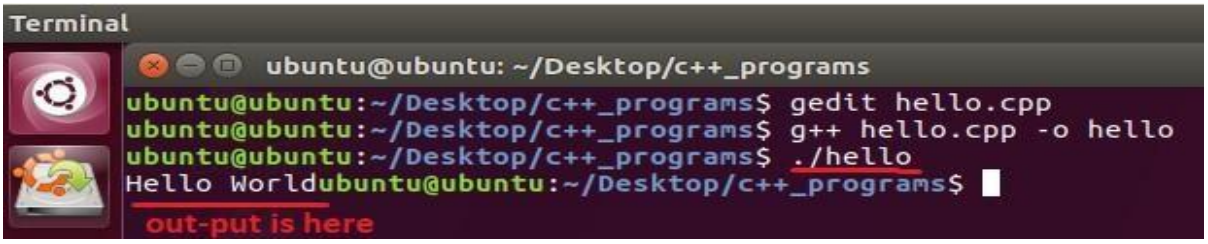
Write the following code



Now, close the source file and write the following command to compile hello.cpp



To execute the hello executable file, write the following command.



Passing command-line arguments to a C++ program

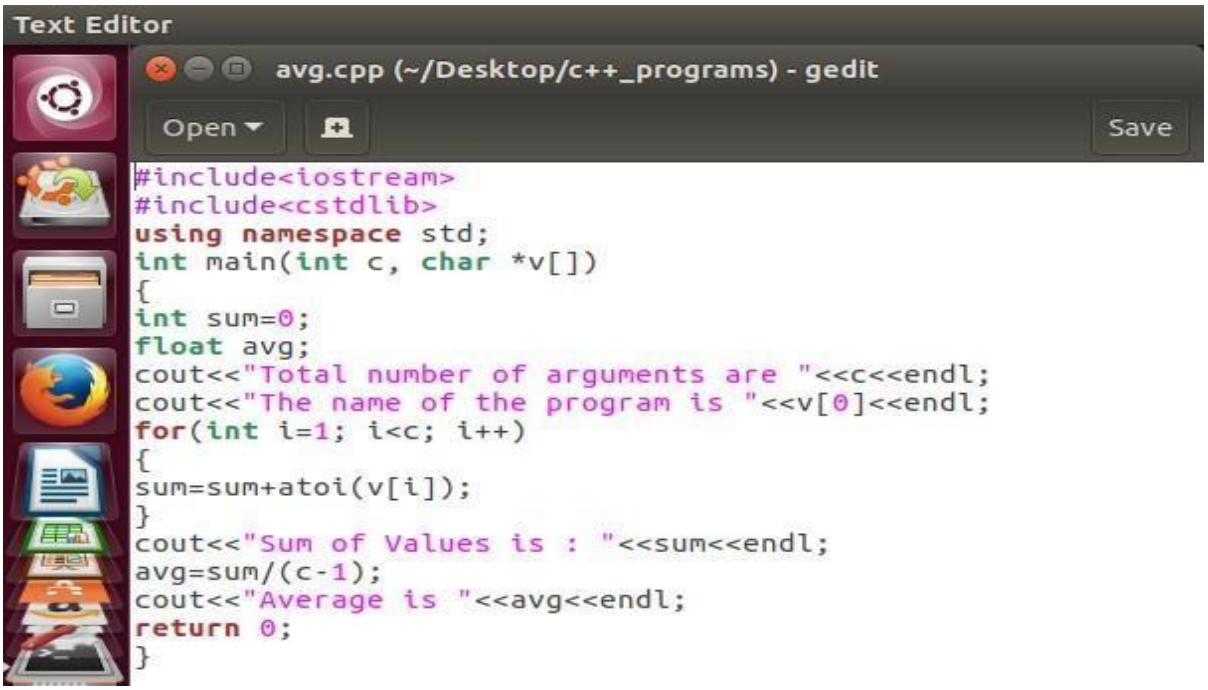
Command line argument is a parameter supplied to the program when it is invoked. Command line argument is an important concept in C++ programming. It is mostly used when you need to control your program from outside. Command line arguments are passed to the main () method.

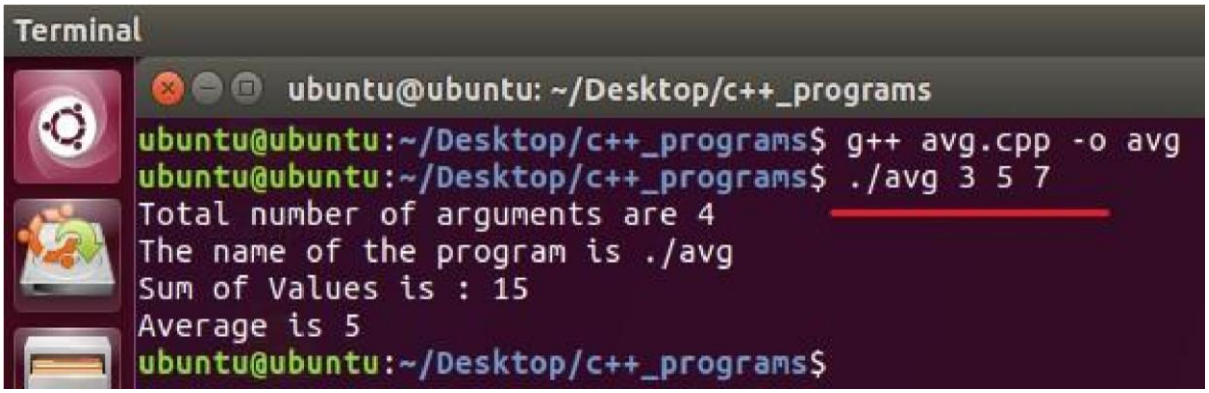
To pass command line arguments, we typically define main () with two arguments: first argument counts the number of arguments on the command line and the second is a pointer array which holds pointers of type char which points to the arguments passed to the program. The syntax to define the main method is

```
int main (int argc, char *argv[])
```

Here, argc variable will hold the number of arguments pass to the program while the argv will contain pointers to those variables. argv[0] holds the name of the program while argv[1] to argv[argc] hold the arguments. Command-line arguments are given after the name of the program in command-line shell of Operating Systems. Each argument separated by a space. If a space is included in the argument, then it is written in “”.

In the following example, we calculate the average of numbers; passed in the command- line. Write the following code and save the file named as avg.cpp. Compile the avg.cpp and create an executable file named avg and execute it. While writing the command to execute avg pass the numbers whose average is required. It is shown below.





2) Lab Activities

Activity 3:

Write a program in C++ that find the maximum and minimum number from an array.

3) Graded Lab Tasks

Task 1:

Change the program given in Activity 3 such that it accepts the input at command-line.

Task 2:

Write a C++ program that accepts a number as input and find whether it is a palindrom or not

Task 3:

Write a program that creates an array of size 1000 and fills this array with random numbers between 2 and 100; and then it finds how many of these numbers are prime.